

GRAPH

PROBLEM 7

QUESTION:

DS	Session	Graph	Question Information	Level 1	Challenge 87
Question description There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads. A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component. Input The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \dots, n$. Then, there are m lines describing the new roads. Each line has two integers a and b: a new road is constructed between cities a and b. You may assume that every road will be constructed between two different cities. Output Print m lines: the required information after each day. Constraints $1 \leq n \leq 10^5$ $1 \leq m \leq 2 \cdot 10^5$ $1 \leq a, b \leq n$					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>5 6 1 2 2 3 3 1 3 4 4 5 5 4</pre> EXPECTED OUTPUT <pre>4 2 3 3 3 3 2 4 1 5 1 5</pre> Test Case 2 INPUT (STDIN) <pre>5 5 1 3 2 4 3 1 3 2 5 4</pre> EXPECTED OUTPUT <pre>4 2 3 2 3 2 2 4 1 5</pre>					
Mandatory Test Cases Test Case 1 KEYWORD <pre>int join(int i,int j)</pre>					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 2 Test Case 2 TOKEN COUNT 300 Test Case 3 NLOC 40					

SOURCE CODE:

```
#include <stdio.h>

#define MAX_TOWNS 100005

int town_parent[MAX_TOWNS];
int group_size[MAX_TOWNS];
int active_components;
int largest_group = 1;

int get_town_root(int x) {
    if (town_parent[x] == x) return x;
    return town_parent[x] = get_town_root(town_parent[x]);
}

void build_link(int u, int v) {
    int root_u = get_town_root(u);
    int root_v = get_town_root(v);

    if (root_u != root_v) {
        if (group_size[root_u] < group_size[root_v]) {
            int swap_val = root_u; root_u = root_v; root_v = swap_val;
        }
        town_parent[root_v] = root_u;
        group_size[root_u] += group_size[root_v];
        if (group_size[root_u] > largest_group) {
            largest_group = group_size[root_u];
        }
        active_components--;
    }
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    active_components = n;
    for (int i = 1; i <= n; i++) {
        town_parent[i] = i;
        group_size[i] = 1;
    }

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        build_link(u, v);
        printf("%d %d\n", active_components, largest_group);
    }
    return 0;
}
```

OUTPUT:

```
5 6
1 2
2 3
3 1
3 4
4 5
5 4
4 2
3 3
3 3
2 4
1 5
1 5
```

```
5 5
1 3
2 4
3 1
3 2
5 4
4 2
3 2
3 2
2 4
1 5
```